

ULingo

Voice Automated Conversational Assistant for Students

Project Team

Eeman Ijaz 21i-1381
Ahmed Hassan 21i-1758

Session 2021-2025

Supervised by

Mr. Sami Ullah Shah



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.2.1	Data Collection	2
1.2.2	Model Fine-Tuning	3
1.2.3	Personalized Opportunities	3
1.2.4	Figma Design for Mobile App	3
1.2.5	Mobile App Development	3
1.2.6	Testing	3
1.3	User Classes and Characteristics	4
2	Project Requirements	5
2.1	Use-Case	5
2.2	High Level Use Cases	6
2.3	Extended Use Cases	7
2.4	Functional Requirements	11
2.4.1	Data Retrieval	12
2.4.2	Model Finetuning	12
2.4.3	Mobile App Development	12
2.5	Non-Functional Requirements	12
2.5.1	Reliability	12
2.5.2	Usability	13
2.5.3	Performance	13
2.5.4	Security	13
2.6	Domain Model	13
3	System Overview	15
3.1	Architectural Design	16
3.2	Design Models	16
3.3	Data Design	18
3.3.1	Key Data Entities	19
3.3.2	Data Processing and Organization	20

References

21

List of Figures

2.1	Use-Case Diagram for ULingo	5
2.2	Domain Model	14
3.1	Modular Structure	15
3.2	Architectural Diagram	16
3.3	Activity Diagram of Ulingo	17
3.4	Class Diagram of Ulingo	18
3.5	Class Sequence Diagram of Ulingo	19

List of Tables

1.1	User Classes and Characteristics	4
2.1	High Level Use Cases	7

Chapter 1

Introduction

With 230 million people speaking Urdu globally, there is a growing need for smart applications that cater to Urdu speakers [5], allowing them to interact through voice rather than text. While conversational bots have been developed in languages like Chinese, English, German, Korean, and Spanish, a significant language barrier prevents non-speakers from utilising these tools effectively. The handful of languages on which NLP systems are trained and tested are often related and from the same geography, drawn from a few dominant language families, leading to a typological echo chamber [2].

Traditional voice assistants rely on conventional language models, which often lack deep contextual understanding. Although large language models (LLMs) excel in this area, they are primarily designed for text-based interactions. Google and Amazon stress 'multilingual support' and 'multilingual mode'. Despite this capability, usability problems from multilingual users have continued, ranging from issues with text messages being read out using another language's speech synthesizer [4] to total language inflexibility when asking for directions [7]. This project aims to bridge this gap by creating a personalized Urdu voice assistant for students, leveraging advanced language models to provide meaningful and contextually rich interactions. This will empower Urdu-speaking students to access opportunities and handle finances more effectively, all through natural voice communication.

1.1 Problem Statement

Conventional voice assistants rely on traditional language models to discern user intent and respond to their queries, leading to interactions that often lack a broader contextual understanding, an area in which Large Language Models (LLMs) excel. However, current LLMs are largely designed for text-based interactions, thus making it unclear how user interactions will evolve if their modality is changed to voice [6]. While commercial

voice assistants such as Alexa and Siri rely on traditional language models to process user requests, they mainly use rule-based keyword recognition mechanisms to determine user intent and fall short of maintaining coherent multi-turn conversations [6]. Hence, it can be deduced that traditional voice assistants do not offer personalized, context-aware interactions [1] vital for effective student support. Multilingual progress in intelligent personal assistants (IPAs) utilising speech interfaces has been particularly slow [3]. Due to language barriers, Urdu-speaking students in Pakistan face significant challenges in accessing digital tools and resources. Most existing voice assistants and chatbots are designed for English and other widely spoken languages, leaving these students underserved. They are unable to participate in educational and professional possibilities due to a lack of accessible technologies. Our solution solves these challenges by creating an Urdu Voice Assistant designed exclusively for students, allowing them to access opportunities and manage their funds in their native tongue.

1.2 Scope

This project focuses on developing a mobile application that serves as a personalised Urdu voice assistant for students in Pakistan. The app's primary goal is to provide voice-based interaction in Urdu, ensuring accessibility for students more comfortable with their native language. A key feature is integrating LinkedIn to deliver curated opportunities tailored to each student's academic and professional needs.

While the app also includes namaz reminders and a student finance budgeting feature, the main emphasis is on offering a seamless Urdu experience and personalized support. The project will ensure the app is user-friendly, secure, and accessible across major mobile platforms, with ongoing updates to enhance functionality.

1.2.1 Data Collection

Gather relevant datasets for training and fine-tuning the Urdu language model and collecting user requirements for app features.

1. Collect Urdu language datasets for NLP.
2. Gather user feedback and requirements.
3. Preprocessing the collected Urdu audio and text data.

1.2.2 Model Fine-Tuning

Fine-tunes the LLaMA 3.1 model to enhance its performance for Urdu language processing.

1. Use of Python and PyTorch for model training.
2. Integration with Hugging Face for managing pre-trained models and datasets.

1.2.3 Personalized Opportunities

Integrate the various components of the app, including the fine-tuned LLaMA 3.1 model, backend services, and third-party APIs.

1. API integration with LinkedIn to retrieve personalised job and internship listings
2. API integration with Google Maps to get current location of students for namaz reminders.

1.2.4 Figma Design for Mobile App

Develop the visual and interactive design of the mobile app using Figma, focusing on a user-friendly interface.

1. Design wireframes and prototypes.
2. Create a cohesive UI/UX for Urdu-speaking users.

1.2.5 Mobile App Development

Develop the mobile application using Flutter, implementing all the designed features and ensuring smooth functionality across platforms.

1. Implement voice interaction, text display features.
2. Ensure cross-platform compatibility.

1.2.6 Testing

Ensures the app's functionality and performance are refined through continuous testing cycles during development.

1. Unit testing to validate individual components for expected behavior.
2. Integration testing to ensure different modules work together without conflicts.
3. User Acceptance Testing (UAT) for users to engage in real-world scenarios to ensure the app meets their needs.

1.3 User Classes and Characteristics

In Table 1.1, the various user categories anticipated to engage with the Urdu voice assistant application are summarized, along with their main attributes. This information will be valuable in developing user-focused functionalities tailored to their individual requirements.

User Class		Characteristics
Undergraduate	Students	These users are tech-savvy, looking for academic and career opportunities, and likely to benefit from stress management and budgeting tools.
Graduate Students		Focused on advanced career opportunities and research, these users may also seek tools for stress management and financial planning.
Educational	Administrators	Oversee the integration of university email systems, ensuring students receive relevant opportunities. They also manage institutional data.
Career Counsellors		Interested in guiding students towards using LinkedIn effectively and leveraging the app's personalized career resources.

Table 1.1: User Classes and Characteristics

Chapter 2

Project Requirements

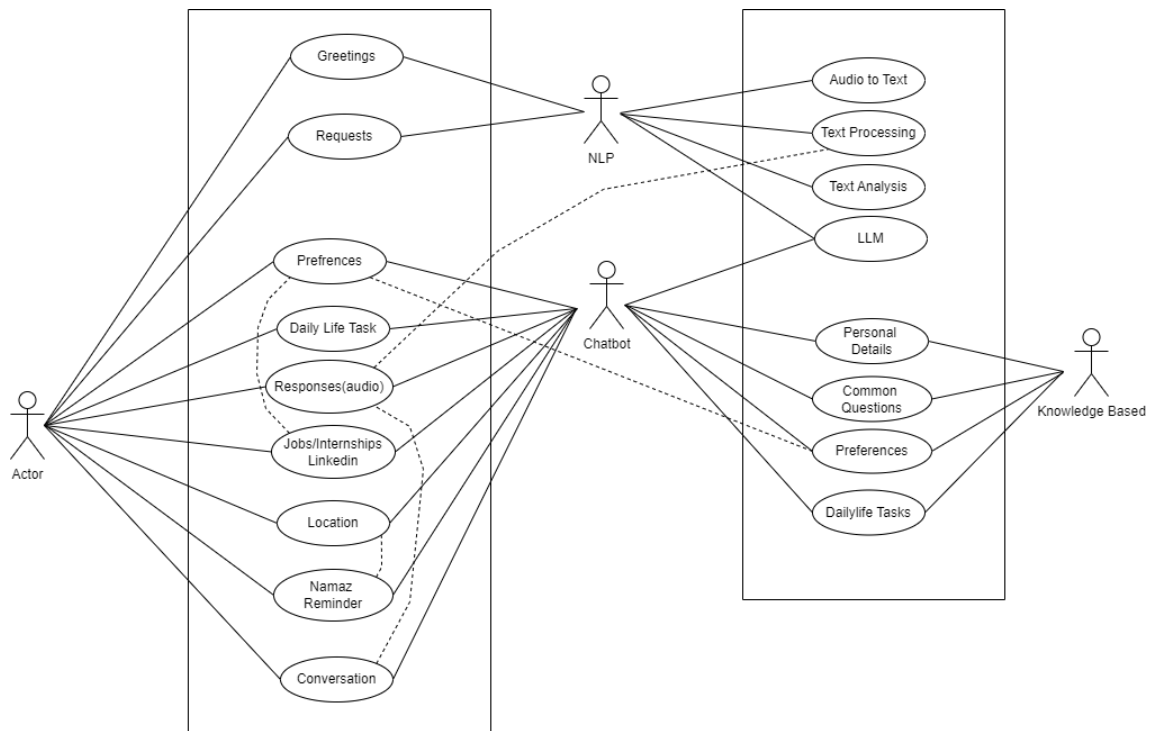


Figure 2.1: Use-Case Diagram for ULingo

2.1 Use-Case

The primary interactions between the user and the voice assistant application are depicted in the use case diagram shown in Figure 2.1. It starts with the system initiating communication in a user-friendly manner by saying **Greetings**. Users can then make various **Requests**, which the system processes, ranging from simple queries to complex commands.

Additionally, the system enables users to set and retrieve **Preferences** to customize their interaction experience based on individual needs.

The system handles **Everyday Tasks**, assisting users in efficiently managing their daily schedules. It offers auditory responses through **Audio Feedback**, improving user engagement by creating more interactive and accessible interactions. In terms of career growth, it utilizes LinkedIn's API to retrieve **Job/Internship** opportunities that are customized to the user's profile.

Furthermore, the app makes use of the user's **Location** to provide localized services like weather updates and navigation, as well as offering **Namaz Reminders** tailored to the user's location and settings to facilitate religious observances. The **Conversation** function demonstrates the system's ability to participate in dynamic interactions, learning from user inputs in order to enhance the quality of responses over time.

2.2 High Level Use Cases

Use Case	Description	Actors	Subsystem Involved
Submit Request	User interacts with the chatbot by submitting a query or command.	User	Chatbot, NLP
Speech Recognition	Converts spoken user input into text for further processing.	User, Speech-to-Text System	NLP
Intent Recognition	The NLP system identifies the user's intent based on the text input.	User, NLP	NLP
Entity Extraction	Extracts key information such as dates, names, or locations from the user's input.	User, NLP	NLP
Query Knowledge Base	The NLP or chatbot queries the knowledge base to retrieve relevant information (e.g., jobs).	Chatbot, Knowledge Base	Knowledge Base
Generate Response	Based on NLP and knowledge base data, the chatbot generates a response.	Chatbot, NLP	Response Generation
Conversation Tracking	Manages the conversation flow, ensuring context is maintained throughout multiple interactions.	Chatbot	Conversation Management

Text-to-Speech	Converts textual responses from the chatbot into audio format for the user.	User, Text-to-Speech System	NLP, Response Generation
View Conversation History	Allows the system to access previous conversations to maintain context.	Chatbot, Conversation History	Conversation Management

Table 2.1: High Level Use Cases

2.3 Extended Use Cases

Use Case 1: Submit Request

Scope: Chatbot System

Level: User Interaction

Primary Actors: User

Stakeholders and Interests: Users want to submit queries or commands to the system.

Success Guarantee: Request successfully received and processed by the system.

Main Success Scenario:

1. User sends a request.
2. System accepts and processes the request.

Extensions:

- Request contains incomplete or invalid data.

Special Requirements: Low latency and accurate parsing.

Technology: NLP, Chatbot System

Frequency of Occurrence: Frequent

Use Case 2: Speech Recognition

Scope: NLP Module

Level: Low-level Processing

Primary Actors: Speech-to-Text System

Stakeholders and Interests: Users expect accurate conversion of speech to text.

Success Guarantee: Audio is accurately transcribed to text.

Main Success Scenario:

1. Speech input is accurately processed and converted to text.

Extensions:

- Poor quality audio, noisy background.

Special Requirements: High accuracy of transcription.

Technology: NLP, Speech Recognition

Frequency of Occurrence: Occasional

Use Case 3: Intent Recognition

Scope: NLP Module

Level: Low-level Processing

Primary Actors: NLP System

Stakeholders and Interests: Users expect the system to accurately understand their intents.

Success Guarantee: Intent is correctly identified from input.

Main Success Scenario:

1. User sends a query.
2. NLP system identifies intent based on query.

Extensions:

- Ambiguous or unclear intent in the user request.

Special Requirements: NLP model accuracy.

Technology: NLP Model

Frequency of Occurrence: Frequent

Use Case 4: Entity Extraction

Scope: NLP Module

Level: Low-level Processing

Primary Actors: NLP System

Stakeholders and Interests: NLP system extracts necessary entities for response processing.

Success Guarantee: Relevant entities like names, dates extracted correctly.

Main Success Scenario:

1. NLP system extracts key entities such as dates, locations, names.

Extensions:

- Entities not clearly specified by the user.

Special Requirements: Named Entity Recognition (NER)

Technology: NLP Model

Frequency of Occurrence: Frequent

Use Case 5: Generate Response

Scope: NLP Module

Level: Low-level Processing

Primary Actors: NLP, Chatbot System

Stakeholders and Interests: Users expect a meaningful response based on their input.

Success Guarantee: Response is generated based on user's input.

Main Success Scenario:

1. Chatbot generates a response based on NLP output and knowledge base.

Extensions:

- Response fails to match user's expectations.

Special Requirements: Response relevance and accuracy.

Technology: NLP, Chatbot System

Frequency of Occurrence: Frequent

Use Case 6: Query Knowledge Base

Scope: Chatbot System

Level: Subsystem Interaction

Primary Actors: Chatbot, Knowledge Base

Stakeholders and Interests: Retrieve domain-specific information effectively.

Success Guarantee: Information from the knowledge base is retrieved accurately.

Main Success Scenario:

1. Chatbot queries the knowledge base for information related to jobs or tasks.
2. The system retrieves and provides the required information.

Extensions:

- No relevant data in the knowledge base.

Special Requirements: Rich domain knowledge.

Technology: Knowledge Base

Frequency of Occurrence: Frequent

Use Case 7: Conversation Tracking

Scope: Conversation System

Level: Context Management

Primary Actors: Chatbot

Stakeholders and Interests: Ensure continuity and relevance in ongoing user interactions.

Success Guarantee: System maintains conversation history and context.

Main Success Scenario:

1. Chatbot maintains conversation flow and remembers context from previous exchanges.

Extensions:

- Context lost due to user confusion or unclear input.

Special Requirements: Persistent conversation memory.

Technology: Chatbot, NLP System

Frequency of Occurrence: Occasional

Use Case 8: Text-to-Speech

Scope: NLP Module

Level: Output Interaction

Primary Actors: Text-to-Speech System

Stakeholders and Interests: User expects responses in audio format to enhance accessibility.

Success Guarantee: Text is successfully converted into speech.

Main Success Scenario:

1. Generated text is converted into speech and played to the user.

Extensions:

- Text-to-speech system malfunctions or provides unclear audio output.

Special Requirements: High-quality text-to-speech generation.

Technology: NLP, Text-to-Speech System

Frequency of Occurrence: Occasional

Use Case 9: View Conversation History

Scope: Chatbot System

Level: Context Retrieval

Primary Actors: Chatbot, User

Stakeholders and Interests: User or system needs to check previous conversations for reference or continuity.

Success Guarantee: Chatbot retrieves and displays previous conversation history accurately.

Main Success Scenario:

1. User checks the history of interactions with the bot.
2. The system provides a complete and accurate history.

Extensions:

- Conversation history not available or incorrectly retrieved.

Special Requirements: Persistent storage.

Technology: Chatbot System

Frequency of Occurrence: Rare

2.4 Functional Requirements

The functional requirements define the core operations that the Urdu voice assistant app must fulfill. These include data collection fine-tuning a machine learning model for Urdu language processing, personalizing student opportunities, and creating a user-friendly mobile interface. The system also needs to integrate external services, including APIs, to enhance functionality, like retrieving career opportunities from LinkedIn and location-based services for prayer reminders.

2.4.1 Data Retrieval

1. The system shall collect data from user inputs through the mobile app.
2. The system shall fetch tailored opportunities from LinkedIn based on the user's preferences.
3. The system shall retrieve the current location of user through Google Maps API.

2.4.2 Model Finetuning

1. The system shall fine-tune the LLaMA 3.1 model to improve Urdu language understanding.
2. The system shall process user inputs and queries in Urdu, generating accurate responses based on the fine-tuned model.

2.4.3 Mobile App Development

1. The system shall authenticate the user during login.
2. The system shall converse with the user in Urdu (Nastaliq).
3. The system shall interact with the user via voice and display chat in text.
4. The system shall provide dark and light modes in the app.

2.5 Non-Functional Requirements

The app's non-functional requirements address its quality, including performance, usability, reliability, and security. It must ensure high reliability with minimal downtime, provide an intuitive and responsive user experience, and guarantee data protection. Security measures should prevent unauthorized access, and usability should prioritize ease of use for Urdu-speaking students with varying technical skills.

2.5.1 Reliability

The Urdu voice assistant app must have a high degree of reliability. Failures could occur in understanding voice commands or in providing accurate recommendations. Mean Time

Between Failures (MTBF) will be used to measure reliability, and error detection strategies will involve logging errors and providing fallback mechanisms. Correction strategies include model retraining and frequent software updates.

2.5.2 Usability

The app's usability will focus on ease of learning and interaction for students, especially those from non-technical backgrounds. Features such as voice interaction and personalized recommendations should be intuitive and error recovery easy. The UI/UX design will prioritize accessibility, especially for Urdu-speaking users.

2.5.3 Performance

The performance goals of the app will ensure that 95% of responses are delivered within 3 seconds of user input, regardless of the mobile platform. This includes fetching personalized job opportunities and location-based reminders like namaz timings.

2.5.4 Security

Security measures will protect user data such as LinkedIn and university email credentials. The system must resist unauthorized access, with encryption used to secure sensitive data. The app will also ensure proper API authentication and user session management to prevent data breaches.

2.6 Domain Model

The domain model provides a structured representation of the data handling and processing mechanisms within the voice assistant system, divided into several key components:

- **Audio to Text:** Converts spoken inputs into text format, facilitating further processing.
- **Text Processing:** Involves refining the converted text to ensure it is ready for analysis, including correcting errors and formatting.
- **Text Analysis:** Examines the text to understand context or intent, crucial for accurately responding to user queries.

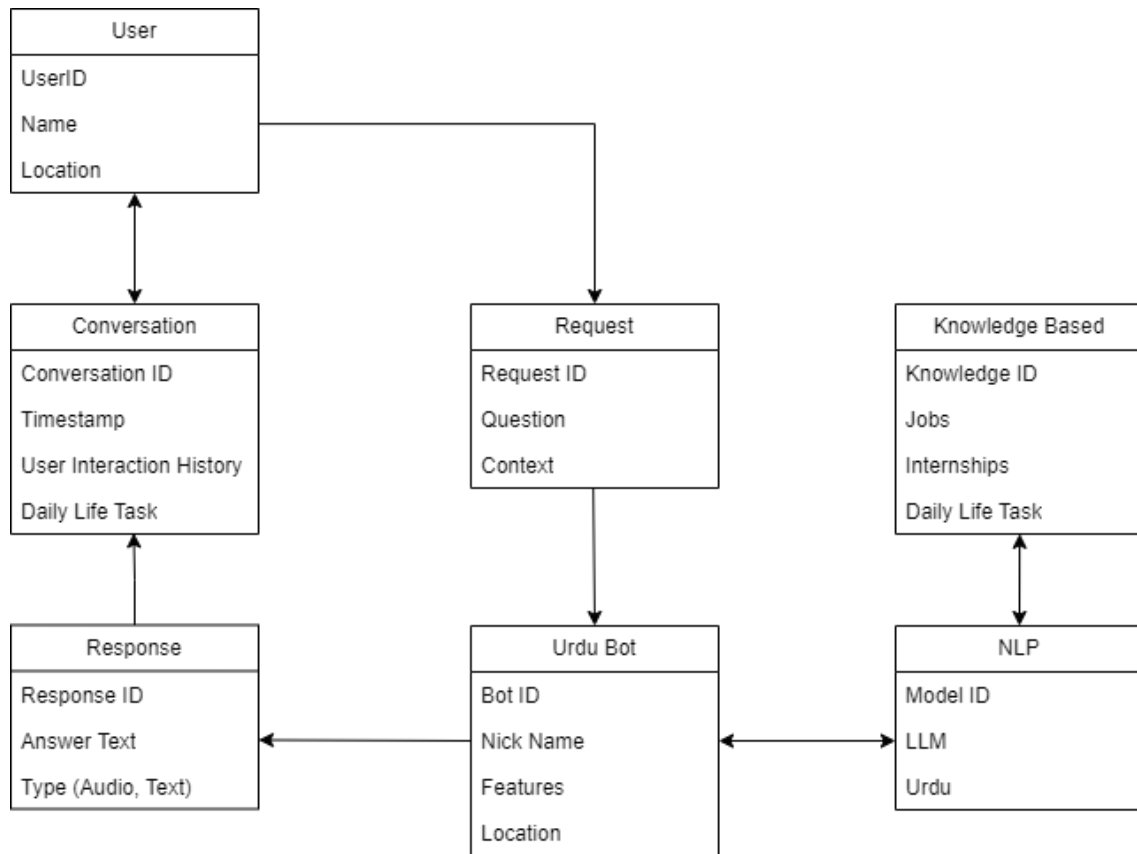


Figure 2.2: Domain Model

- **LLM (Large Language Model)**: Uses advanced algorithms to interpret and respond to user inputs intelligently.

On the user data side, the system manages:

- **Personal Details**: To personalize interactions.
- **Common Questions**: Cataloged to quicken response times for frequently asked queries.
- **Preferences**: Utilized to tailor the system's operations to the user's likes.
- **Daily Life Tasks**: Provides assistance in managing daily schedules effectively.

Chapter 3

System Overview

This Urdu voice assistant mobile application project aims to assist students in effectively managing their academic and personal lives. Through voice commands in Urdu, the app enables students to receive personalized career opportunities, financial budgeting assistance, and reminders, including Namaz timing alerts based on their location. The design emphasizes the integration of LinkedIn for career opportunities and Google Maps API for location-based services. The system's architecture involves gathering Urdu datasets, refining LLaMA 3.1, and creating a user-friendly mobile interface using Flutter.

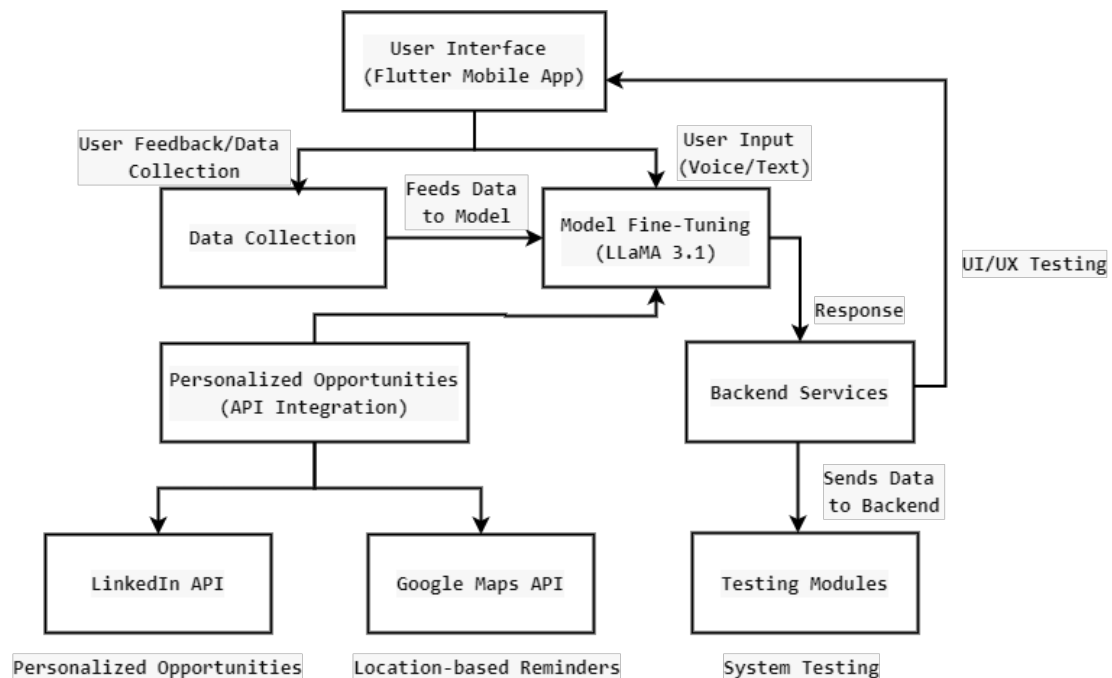


Figure 3.1: Modular Structure

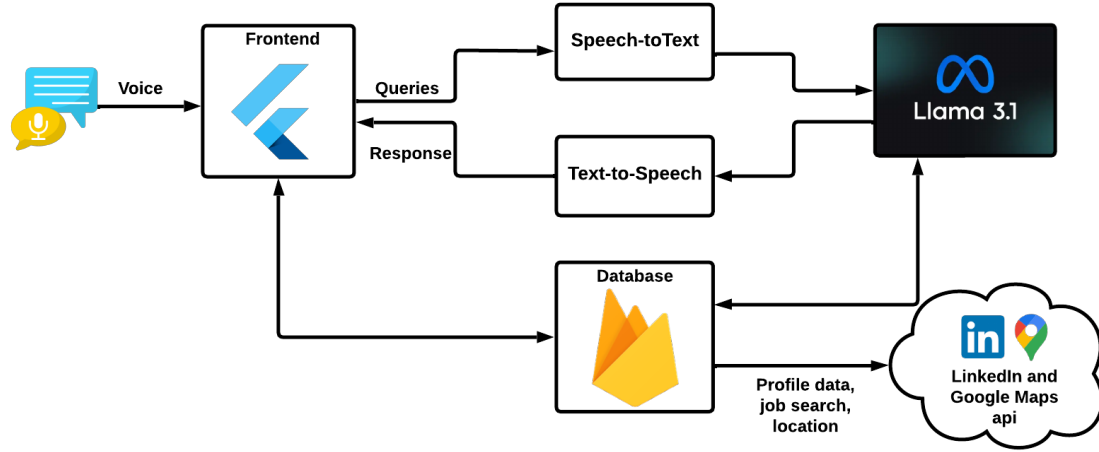


Figure 3.2: Architectural Diagram

3.1 Architectural Design

The architecture of a mobile application shown in Figure 3.1 and 3.2 is developed using Flutter is illustrated in the modular structure diagram, showcasing the flow from user interaction to data processing and response generation. It encompasses key modules such as the User Interface for input collection, and Data Collection for directing inputs to the Model Fine-Tuning module powered by LLaMA 3.1 for intelligent processing. The functionality is enhanced by API Integration with LinkedIn and Google Maps, offering personalized opportunities and location-based services. Backend Services are in place to support system operations, while Testing Modules ensure the system's robustness through comprehensive UI/UX and system tests.

3.2 Design Models

We have utilized an object-oriented (OO) development approach to organize our system for this project. This method is well-suited for the intricacy of the voice assistant, as it necessitates reusable, modular components like voice input processing, API integration, and response generation. The OO method facilitates encapsulation, enabling the system to be scalable and easily maintained. Detailed descriptions of each design model implemented are provided below.

Figure 3.3 shows a step-by-step flow of actions from both the User and the system perspective. The User initiates the process by starting a conversation, after which the system greets them and requests input. Depending on the query, the system either retrieves job opportunities through the LinkedIn API or obtains prayer reminders using the Google

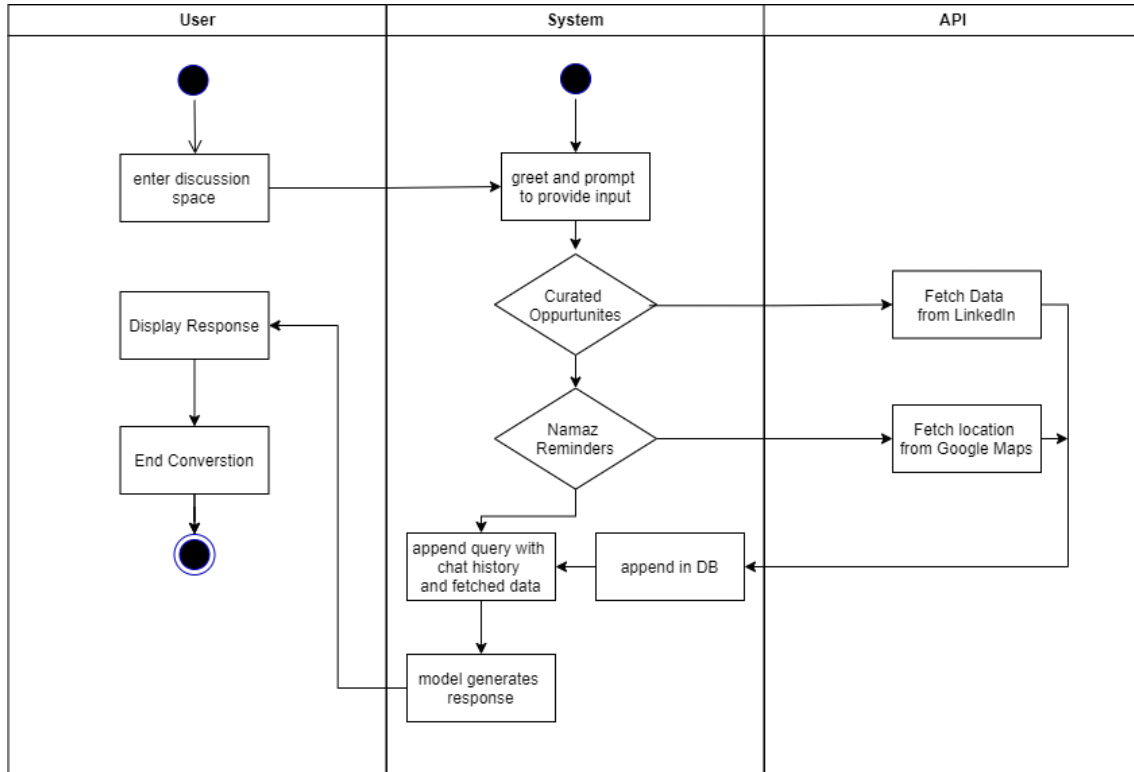


Figure 3.3: Activity Diagram of Ulingo

Maps API. The Model then processes this data, adds it to the Database, and subsequently provides a response to the User. Finally, the interaction either continues or concludes based on the User's input.

Figure 3.4 is the class diagram for Ulingo. It represents the interaction between the User and various components of the voice assistant system. The User begins a conversation with the Voice Assistant, for example, by inquiring about job opportunities or requesting reminders for prayers. The Voice Assistant communicates with various subsystems, including the Model for processing and comprehending queries, the LinkedIn API, and the Google Maps API. Subsequently, the information is stored in the Database for future reference, and a response is then provided to the User.

In Figure 3.5 (Class Level Sequence Diagram), we see the more specific interaction between different classes and objects. The User communicates with the Voice Assistant, which then sends the query to the Model for language processing. If the request involves job data, the JobAPI is called to fetch opportunities. For location-based services, the LocationAPI is triggered. Once the necessary data is retrieved, it is stored in the Database, and the User is provided with a response based on the query.

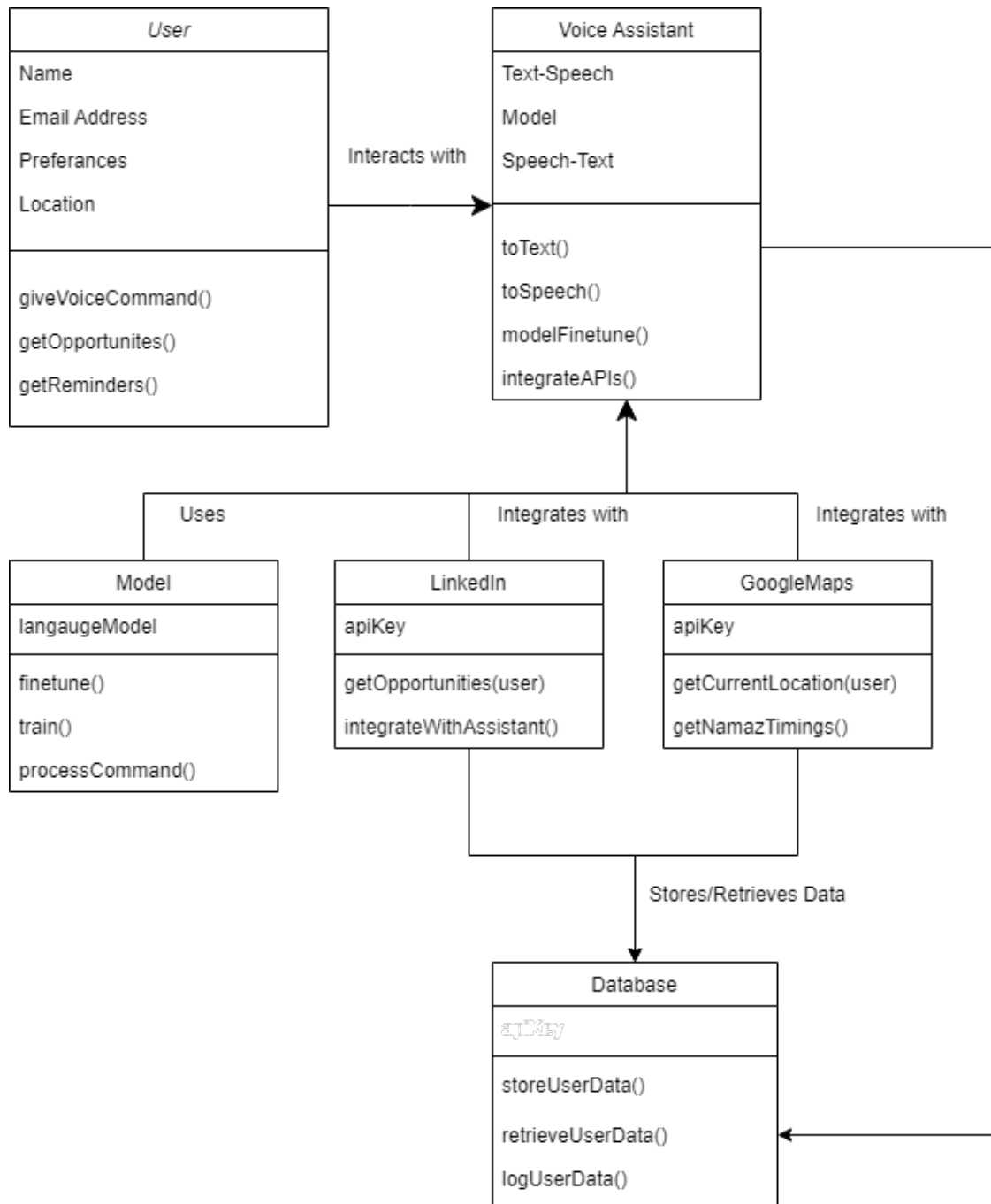


Figure 3.4: Class Diagram of Ulingo

3.3 Data Design

The data structures in this system are created from the information domain using Firebase, which is Flutter's backend solution. Firebase is a NoSQL database hosted in the cloud, storing data in a flexible, document-oriented format arranged into collections.

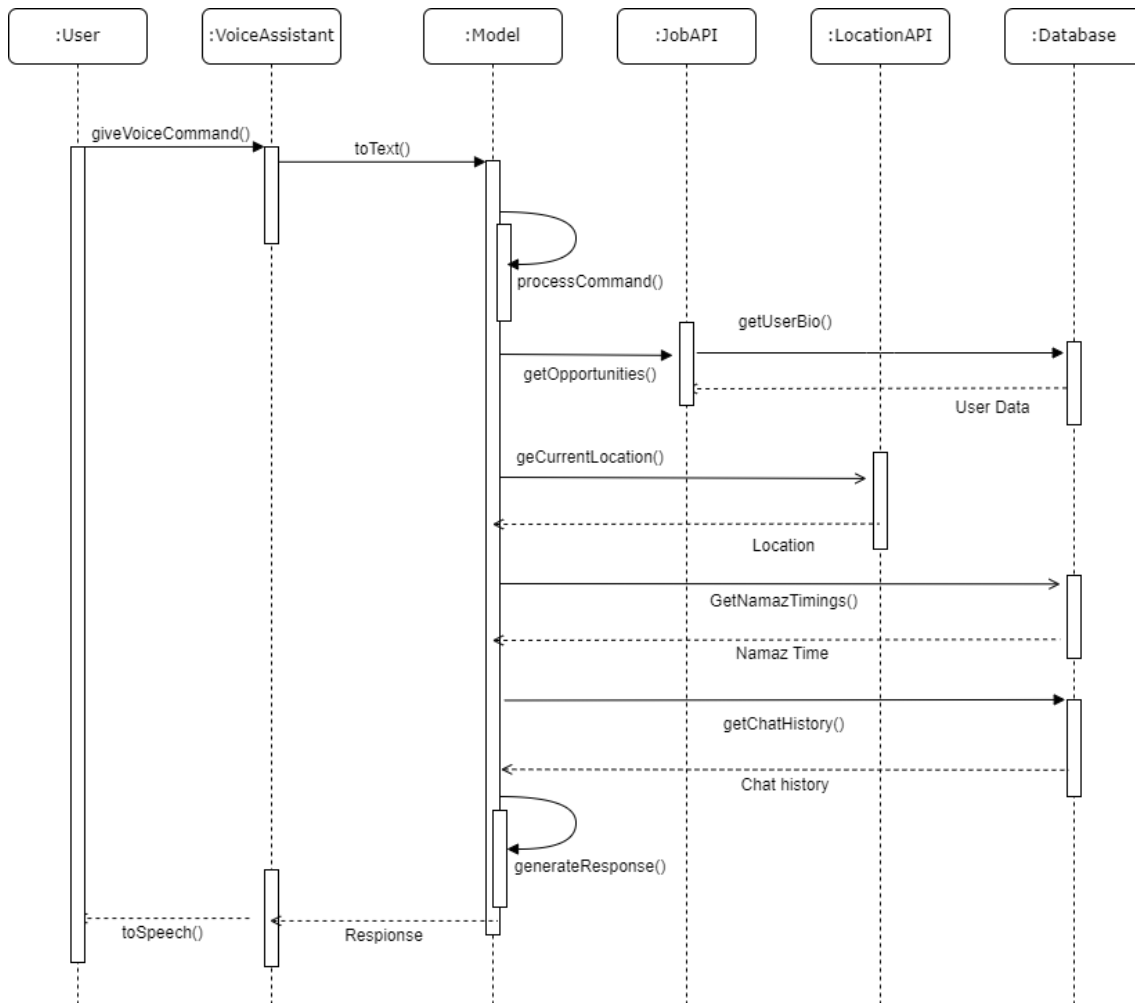


Figure 3.5: Class Sequence Diagram of Ulingo

3.3.1 Key Data Entities

- **User Data:** Contains user profiles, preferences, LinkedIn credentials, and location data. This information is stored in Firebase Firestore for real-time updates and retrieval.
- **Job Opportunities:** These are fetched from LinkedIn's API and temporarily stored to curate personalized job suggestions based on user preferences.
- **Voice Inputs and Outputs:** User voice inputs and system-generated responses are processed by the model and stored in Firestore to improve the model's performance and maintain interaction history.
- **Namaz Reminders:** Based on the user's location, fetched using the Google Maps API, prayer reminders are stored and used to notify the user at appropriate times.
- **API Keys:** Sensitive information, such as API keys for LinkedIn and Google Maps,

is securely handled using Firebase's environment variable storage to ensure safety and security.

3.3.2 Data Processing and Organization

Firebase supports real-time synchronization of data, making it ideal for the interactive nature of this app. The following key processes are implemented:

- **Authentication:** Firebase Authentication is used to securely manage user login credentials and session management, ensuring user data is protected.
- **Data Retrieval:** Data is retrieved from Firebase and external APIs (LinkedIn, Google Maps), processed by the machine learning model, and stored in Firestore to generate personalized recommendations and responses.
- **Real-Time Syncing:** Firebase ensures that changes to user data, interactions, and preferences are synced in real-time across the user's devices.

Bibliography

- [1] Smit Desai and Jessie Chin. Hey google, can you help me learn? In *Proceedings of the 3rd Conference on Conversational User Interfaces*, CUI '21, New York, NY, USA, 2021. Association for Computing Machinery.
- [2] Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. The state and fate of linguistic diversity and inclusion in the NLP world. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6282–6293, Online, July 2020. Association for Computational Linguistics.
- [3] Amanda Kann. Voice assistants have a plurilingualism problem. In *Proceedings of the 4th Conference on Conversational User Interfaces*, pages 1–5, 2022.
- [4] Taylor Kerns. Google offers a silly solution for multilingual android auto users: Stick to one language, 2021.
- [5] Areeba Khalid, Malik Daler Ali Awan, Nadeem Iqbal Kajla, Amnah Firdous, Hafiz Muhammad Sanaullah Badar, and Malik Muhammad Saad Missen. Audio-to-text urdu chatbot using deep learning algorithms rnn and wav2vec2. *Journal of Computing & Biomedical Informatics*, 2024.
- [6] Amama Mahmood, Junxiang Wang, Bingsheng Yao, Dakuo Wang, and Chien-Ming Huang. Llm-powered conversational voice assistants: Interaction patterns, opportunities, challenges, and design guidelines. *arXiv preprint arXiv:2309.13879*, 2023.
- [7] Chad Taylor. Siri, alexa, and other voice assistants struggle with bilinguals, 2020.